

Reviewer Pack — Minimal p-Adic Order and Frege Proof Depth Inequality

Entry eec41c045657 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 03:49:25 UTC

Minimal p-Adic Order and Frege Proof Depth Inequality

Entry ID: eec41c045657 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 03:49:25 UTC

1. Conjecture

Field A (mathematical branch): p-Adic Number Theory **Field B** (complexity object): Boolean Satisfiability (Frege Proof Complexity)

Statement:

For any given CNF φ with n variables, the minimum p-adic order of a polynomial representing φ is at least some constant c times its Frege proof depth $d(\varphi)$, i.e., $\text{ord}_p(\text{poly}(\varphi)) \geq c * d(\varphi)$.

Rationale (proposer's reasoning):

p-Adic numbers offer a non-Archimedean metric that could provide a different perspective on complexity measures. The p-adic order of a polynomial may reflect the complexity of manipulating its coefficients, which is related to proof length in Frege systems.

Taxonomy category: p-Adic_Analysis × Boolean_Satisfiability (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d0a077310d709ac5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a CNF φ with n variables, if the minimum p-adic order of its polynomial representation $\text{poly}(\varphi)$ is less than or equal to c times its Frege proof depth $d(\varphi)$, then support for the conjecture is provided.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "p-adic number theory" AND "Frege proof complexity" AND minimal order" - "CNF" AND "p-adic order" AND Frege depth" - "polynomial representation" AND p-adic AND Frege proof complexity

Top relevant hits considered: - [http://arxiv.org/abs/math/0304173v3] Quantum Groups, the loop Grassmannian, and the Springer resolution - [s2:2110.15734] The extended Smale's 9th problem - On computational barriers and paradoxes in estimation, regularisation, computer-assi - [s2:d0d995160a399bb372267bc86b388a9b87d40696] THE RANKS OF $m \times n \times (mn^2)$ TENSORS *

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def frege_proof_depth(cnf):
        depth = 0
        for clause in cnf:
            if len(clause) == 1:
                depth += 1
            else:
                depth += 2
        return depth

    def p_adic_order(poly, p):
        order = 0
        while poly % p == 0:
            poly //= p
            order += 1
        return order

    n = random.randint(5, 40)
    cnf = generate_cnf(n)
    d_phi = frege_proof_depth(cnf)
```

```

# Construct polynomial representation using p-adic field
p = random.choice([3, 5, 7, 11, 13]) # Choose an odd prime
poly = 0
for clause in cnf:
    term = 1
    for var in clause:
        if var > 0:
            term *= (var + n)
        else:
            term *= (-var)
    poly += term

ord_p_poly = p_adic_order(poly, p)

return {
    "metric_name": "p-adic Order / Frege Proof Depth",
    "metric_value": ord_p_poly / d_phi,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": ord_p_poly >= 0.5 * d_phi, # Example threshold
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [
        int(random.choice([2**i + 3 for i in range(5, 10)])) for _ in range(30)
    ]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

'metric_value': 0.0, 'instances_tested': 1, 'n_max': 21, 'conjecture_holds': False, 'counterexample':
TRIAL: {'metric_name': 'p-adic Order / Frege Proof Depth', 'metric_value': 0.0, 'instances_tested': 1,
TRIAL: {'metric_name': 'p-adic Order / Frege Proof Depth', 'metric_value': 0.016666666666666666, 'ins

```

```

TRIAL: {'metric_name': 'p-adic Order / Frege Proof Depth', 'metric_value': 0.0, 'instances_tested': 1
TRIAL: {'metric_name': 'p-adic Order / Frege Proof Depth', 'metric_value': 0.01, 'instances_tested': 1
TRIAL: {'metric_name': 'p-adic Order / Frege Proof Depth', 'metric_value': 0.0, 'instances_tested': 1
TRIAL: {'metric_name': 'p-adic Order / Frege Proof Depth', 'metric_value': 0.0, 'instances_tested': 1
TRIAL: {'metric_name': 'p-adic Order / Frege Proof Depth', 'metric_value': 0.0, 'instances_tested': 1
TRIAL: {'metric_name': 'p-adic Order / Frege Proof Depth', 'metric_value': 0.015625, 'instances_tested': 1
TRIAL: {'metric_name': 'p-adic Order / Frege Proof Depth', 'metric_value': 0.0, 'instances_tested': 1
TRIAL: {'metric_name': 'p-adic Order / Frege Proof Depth', 'metric_value': 0.0, 'instances_tested': 1
TRIAL: {'metric_name': 'p-adic Order / Frege Proof Depth', 'metric_value': 0.0, 'instances_tested': 1
RESULT: FALSIFIED counterexample="" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test only considers a small range of n (5 to 40) and uses an example threshold for the conjecture, which is not well-defined or justified. The metric does not scale trivially with n , but the number of instances tested is too small to draw robust conclusions.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that the conjecture does not hold for at least one instance tested, as evidenced by the counterexample provided. | next: Further investigation is needed to determine if there are specific conditions under which the conjecture holds or to refine the conjecture itself.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14251
2	preregistration	ollama_remote	glm4:latest	0	0	9554
3	novelty	ollama_remote	glm4:latest	0	0	8387
4	novelty	ollama_remote	glm4:latest	0	0	9273
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	45455
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9659
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8918
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8137
9	critic	ollama_remote	glm4:latest	0	0	19386
10	judge	ollama_remote	glm4:latest	0	0	8950

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 141970 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/eec41c045657.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/eec41c045657.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/eec41c045657.tar.gz` (if generated)